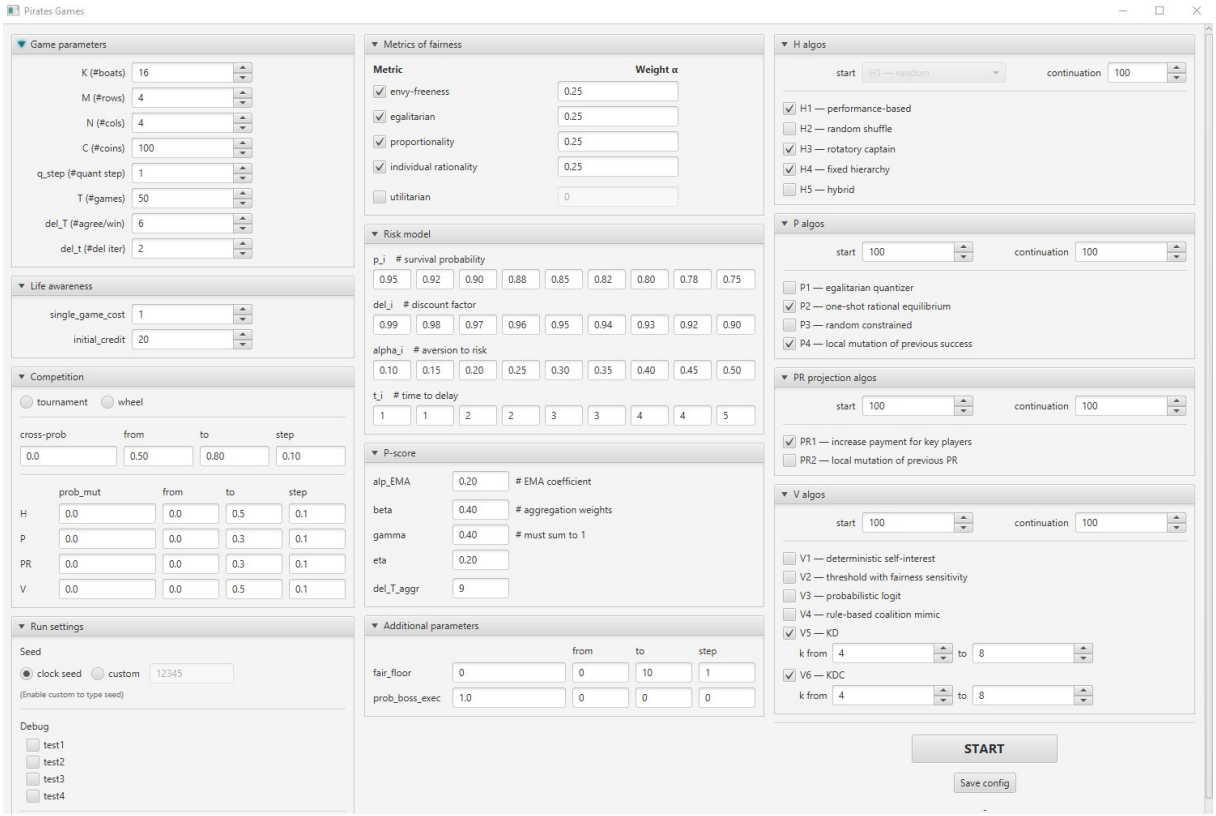
DOKUMENTACJA PROJEKTU

Pirates Game - symulacja strategii, hierarchii i sprawiedliwości

Prototyp aplikacji do konfiguracji, uruchamiania i wizualizacji symulacji gry piratów



Java • JavaFX • FXML • Maven • Jackson

Bartosz Ochnio | 2026

1. Informacje o projekcie

Pirates Game jest rozwijaną aplikacją symulacyjną inspirowaną problemem podziału zasobów pomiędzy piratów. Program pozwala konfigurować populację łodzi, parametry rozgrywek oraz zestawy strategii odpowiedzialnych za hierarchię, przygotowanie propozycji, modyfikację podziału i głosowanie.

W obecnej wersji aplikacja tworzy hierarchie graczy, przeprowadza podstawowe etapy symulacji, oblicza wybrane metryki sprawiedliwości i prezentuje wyniki na wykresach oraz w widoku całej populacji.

Status projektu: prototyp w wersji rozwojowej. Podstawowy przepływ danych i wizualizacje działają, ale część algorytmów, parametrów i interpretacji wyników może wymagać dalszej implementacji oraz walidacji.

Najważniejsze zaimplementowane elementy

- konfiguracja planszy, liczby łodzi, liczby gier i zasobów;
- tworzenie hierarchii piratów;
- wybór wariantów strategii H, P, PR i V;
- konfigurowanie mutacji i mechanizmu konkurencji;
- obliczanie kilku metryk sprawiedliwości;
- wizualizacja wyników pojedynczej łodzi;
- porównanie wyników całej populacji;
- zapisywanie i ponowne wykorzystanie konfiguracji.

Technologie

Technologia	Zastosowanie
Java	logika symulacji i modele danych
JavaFX	interfejs i wizualizacje
FXML	definicja widoków aplikacji
Maven	budowanie projektu i zależności
Jackson	odczyt i zapis konfiguracji w formacie JSON

2. Model symulacji

Symulacja obejmuje populację łodzi rozmieszczonych na planszy M × N. Każda łódź posiada zestaw strategii określających sposób tworzenia hierarchii, przygotowania propozycji, modyfikacji podziału oraz podejmowania decyzji w głosowaniu.

Podstawowy przebieg pojedynczej gry

1. Utworzenie hierarchii - strategia H określa kolejność i role graczy na łodzi.
2. Przygotowanie propozycji - strategia P wyznacza początkowy podział dostępnych monet.
3. Modyfikacja propozycji - strategia PR może zmienić przygotowany podział.
4. Głosowanie - strategia V określa sposób podejmowania decyzji przez graczy.
5. Ocena wyniku - zapisywane są między innymi ostatnia runda oraz wartości metryk sprawiedliwości.
6. Kontynuacja - w kolejnych grach strategie mogą być utrzymywane albo zmieniane zgodnie z konfiguracją.

Rodziny strategii

Symbol	Rola w symulacji
H	tworzenie i zmiana hierarchii
P	generowanie propozycji podziału
PR	modyfikowanie lub projekcja propozycji
V	podejmowanie decyzji w głosowaniu
Kod łodzi: zestaw aktywnych strategii H, P, PR i V tworzy opis zachowania łodzi, prezentowany w dokumentacji projektu jako jej DNA.	

3. Konfiguracja symulacji

Główny formularz grupuje parametry według ich znaczenia. Dzięki temu można osobno ustawić rozmiar populacji, koszty rozgrywek, ryzyko, konkurencję, metryki oraz dostępne strategie.

Grupa	Przykładowe ustawienia
Game parameters	liczba łodzi K, rozmiar planszy M × N, liczba monet C, liczba gier T
Life awareness	koszt pojedynczej gry i początkowy kredyt
Competition	rodzaj konkurencji, krzyżowanie i mutacje strategii
Run settings	ziarno losowości oraz tryby diagnostyczne
Risk model	prawdopodobieństwo przeżycia, dyskontowanie, awersja do ryzyka i czas oczekiwania
P-score	współczynniki agregujące ocenę zachowania
Strategies	aktywne warianty H, P, PR i V oraz warunki ich kontynuacji

Metryki sprawiedliwości

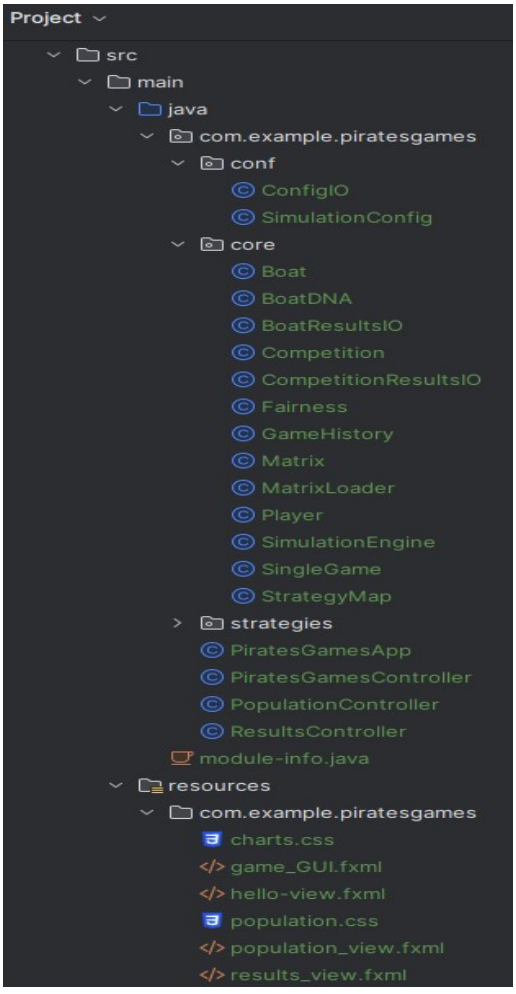
- envy-freeness - ograniczanie sytuacji, w których gracz preferuje przydział innego gracza;
- egalitarian - ocena równości podziału;
- proportionality - ocena proporcjonalności wypłat;
- individual rationality - ocena opłacalności wyniku dla poszczególnych graczy;
- utilitarian - ocena łącznej użyteczności.

Użytkownik może włączać wybrane metryki i przypisywać im wagi. Łączna ocena stanowi agregację aktywnych składników.

4. Struktura projektu

Projekt rozdziela konfigurację, modele symulacji, logikę strategii, kontrolery oraz zasoby interfejsu. Główny silnik koordynuje przebieg gier i przekazuje wyniki do widoków JavaFX.

Element	Odpowiedzialność
conf	modele konfiguracji i zapis ustawień
core	łodzie, gracze, historia, wyniki, macierze i silnik symulacji
strategies	implementacje strategii H, P, PR i V
PiratesGamesController	obsługa głównego formularza
ResultsController	prezentacja wyników pojedynczej łodzi
PopulationController	prezentacja wyników całej populacji
resources	pliki FXML oraz arkusze CSS



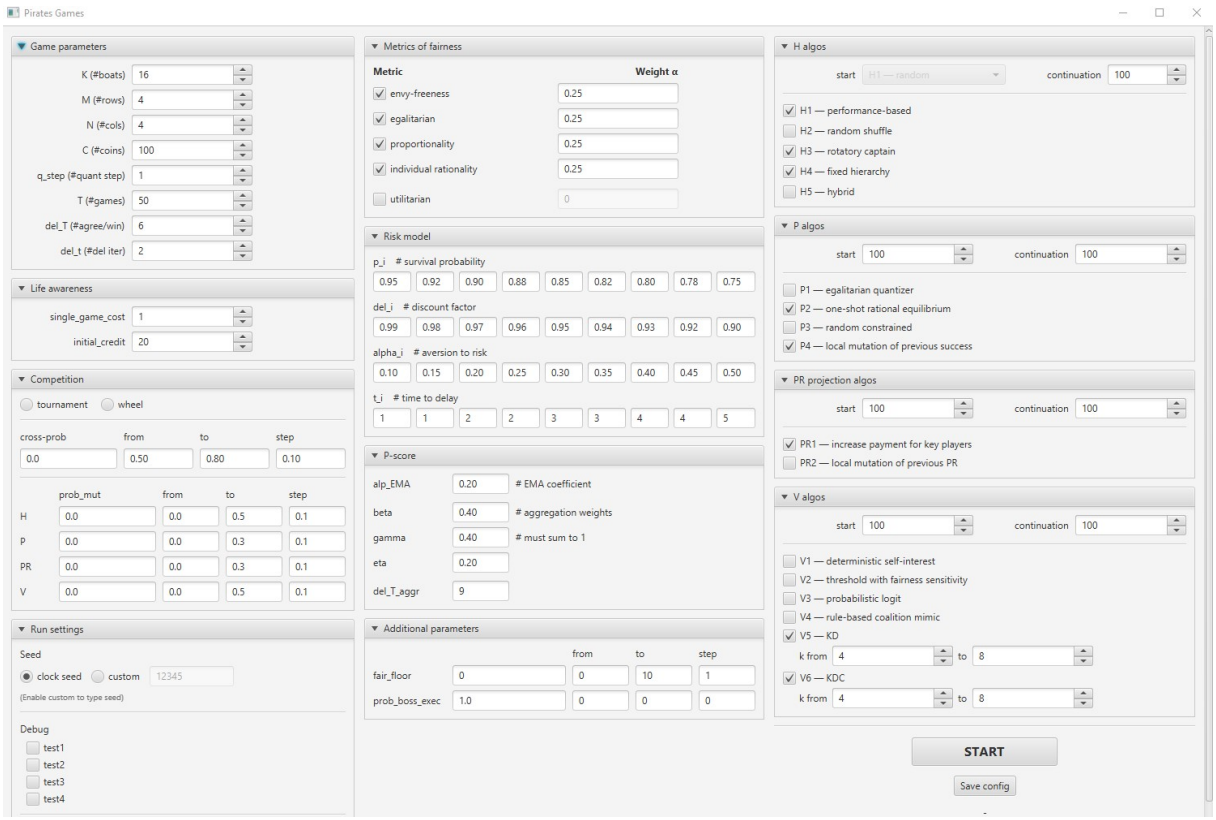
Rysunek 1. Struktura projektu.

Przepływ danych

Konfiguracja jest odczytywana z interfejsu i przekazywana do silnika symulacji. Silnik tworzy obiekty łodzi i graczy, stosuje wybrane strategie, zapisuje historię gier oraz oblicza wyniki. Kontrolery wizualizacji przekształcają te dane w tabele i wykresy.

5. Główny formularz

Główne okno umożliwia przygotowanie konfiguracji bez ręcznej edycji plików. Parametry podzielono na sekcje dotyczące planszy, sprawiedliwości, ryzyka, konkurencji i strategii. Po wybraniu ustawień użytkownik uruchamia symulację przyciskiem START.



Rysunek 2. Formularz konfiguracji symulacji.

6. Wyniki pojedynczej łodzi

Widok wyników pozwala wybrać łódź i przeanalizować jej zachowanie w kolejnych grach. Informacje są przedstawiane na trzech wykresach.



Rysunek 3. Wyniki wybranej łodzi w kolejnych grach.

Wykres	Prezentowana informacja
Last round	runda, w której zakończyła się dana gra
DNA	strategie H, P, PR i V używane w kolejnych grach
Fairness	wartości aktywnych metryk oraz ocena łączna

Taki widok umożliwia zestawienie zmian strategii z przebiegiem gry i wartościami miar sprawiedliwości.

7. Widok populacji

Widok populacji przedstawia stan wszystkich łodzi dla wybranej gry. Dane są rozmieszczone zgodnie z układem planszy, co pozwala porównywać wyniki i strategie pomiędzy łodziami.



Rysunek 4. Widok wyników całej populacji dla wybranej gry.

Prezentowane informacje

- numer ostatniej rundy;
- łączna wartość F total;
- aktywna strategia hierarchii H;
- aktywna strategia propozycji P;
- aktywna strategia głosowania V;
- średnia i najlepsza wartość F total w kolejnych grach.

8. Uruchomienie projektu

Aplikację można uruchomić jako gotowy pakiet dla systemu Windows albo bezpośrednio z kodu źródłowego w środowisku programistycznym.

Uruchomienie gotowej aplikacji w systemie Windows

Gotowa wersja programu zawiera środowisko uruchomieniowe oraz plik app.bat, dlatego nie wymaga instalowania środowiska Java, Mavena ani IntelliJ IDEA.
Aby uruchomić aplikację:

1. Pobierz archiwum zawierające gotową wersję programu.
2. Rozpakuj całą zawartość archiwum do wybranego katalogu.
3. Zachowaj oryginalną strukturę podkatalogów i plików danych.
4. Uruchom plik: app.bat

Nie należy przenosić samego pliku app.bat poza katalog aplikacji. Program korzysta z dołączonego środowiska uruchomieniowego oraz plików danych znajdujących się w pozostałych podkatalogach.

Uruchomienie z kodu źródłowego

Wymagania

Do uruchomienia projektu z kodu źródłowego potrzebne są:

- środowisko Java zgodne z projektem;
- Maven albo dołączony Maven Wrapper;
- IntelliJ IDEA lub inne środowisko obsługujące projekty Maven;
- wymagane pliki danych, konfiguracji i macierzy.

Instrukcja

1. **Otwórz projekt** - zaimportuj w IntelliJ IDEA katalog zawierający plik pom.xml.
2. **Pobierz zależności** - poczekaj na zakończenie synchronizacji projektu Maven.
3. **Uruchom aplikację** - wykonaj główną klasę JavaFX projektu.

Uwaga dotycząca ścieżek: uruchomienie programu przez plik app.bat i bezpośrednio z IntelliJ IDEA może wykorzystywać różne katalogi robocze. W takiej sytuacji te same ścieżki względne mogą wskazywać inne lokalizacje plików. Najprostszym sposobem uruchomienia gotowej wersji jest użycie pliku app.bat bez zmieniania struktury rozpakowanego katalogu.